



Computer Networks

Assignment - 02

OSAMA MUHAMMAD AMIR AYUB ABDULLAH SAJID	B23F0333AI079 B23F0767AI193 B23F0001AI062
DEPARTMENT	CS AND IT
PROGRAM	BS AI
SECTION	AI BLUE

Design of Simple Web Protocol (SWP): An Application Layer Protocol for Web Servers.

Table of Contents

Introduction (Brief overview of protocols designed and its purpose).....	3
Application Functional Requirements.....	3
Application NON-Functional Requirements.....	4
Application Architecture	5
System Overview:	5
Component Architecture.....	6
Connection Flow	6
Message Types.....	6
Request message.....	7
Response Message.....	7
Message Format	7
Request Message Format.....	7
Response Message Format	10
Any Additional Information for the Developers	13

Introduction (Brief overview of protocols designed and its purpose)

This document outlines the design of the Simple Web Protocol (SWP), a lightweight, text-based request–response protocol for web server communication. SWP is built on **TCP/IP** and follows a client-server model where clients request resources and servers return content or status messages.

Designed for clarity and educational purposes, SWP focuses on simplicity, readability, and easy implementation. It includes core features such as structured message formats, standard status codes, basic content negotiation, and consistent error handling. The protocol remains stateless, scalable, and extensible, forming a clean foundation for web services while allowing future enhancements like security support or advanced content delivery.

This specification provides the essential functional requirements, architectural choices, and implementation guidelines needed for developers to build SWP-compliant applications effectively.

Application Functional Requirements

- **Resource Retrieval:** Serve static files (HTML, CSS, JavaScript, images) from server storage
- **Request Methods Support:**
 - GET: Retrieve resource representation
 - HEAD: Retrieve only headers without body
 - POST: Submit data to server for processing
- **Handle HTTP Requests:** The server must be able to receive and process incoming HTTP requests (e.g., GET, POST) from clients (web browsers)
- **Serve Static Content:** It must retrieve and send requested files (HTML pages, images, CSS files, etc.) from a specified directory on the server's file system

- **Route Requests:** It must direct incoming requests to the appropriate handler or file based on the URL or request parameters.
- **Error Handling:** When a requested file or resource is not found, the server must generate and return an appropriate HTTP error response (e.g., a "404 Not Found" page)
- **Manage Connections:** It should establish and terminate network connections (TCP/IP) with clients in accordance with the HTTP protocol
- **Logging:** The system must record details of requests and errors in log files for monitoring and debugging purposes

Application NON-Functional Requirements

- **Performance:**
 - **Response Time:** The server should process and respond to typically GET requests within a specified time frame (e.g., under 1 second)
 - **Throughput/Capacity:** The system should be able to handle a certain number of concurrent users or requests without significant performance degradation (e.g., 5,000 simultaneous users)
- **Security:**
 - **Data Protection:** User data and communications (if applicable) must be encrypted during transmission using protocols like HTTPS/SSL/TLS
 - **Access Control:** The server should prevent unauthorized access to restricted files or administrative interfaces
 - **Resilience:** Security measures should be in place to protect against common web vulnerabilities (e.g., DDoS attacks, injection attacks)
- **Availability/Reliability:**
 - **Uptime:** The web server should be highly available, targeting an uptime percentage (e.g., 99.9%)
 - **Mean Time Between Failures (MTBF):** The server software should operate reliably for long periods with minimal failures

➤ **Scalability:**

- The system should be designed to handle growth (e.g., an increasing number of users or files) by adding resources (horizontal or vertical scaling) without major architectural changes.

➤ **Maintainability:**

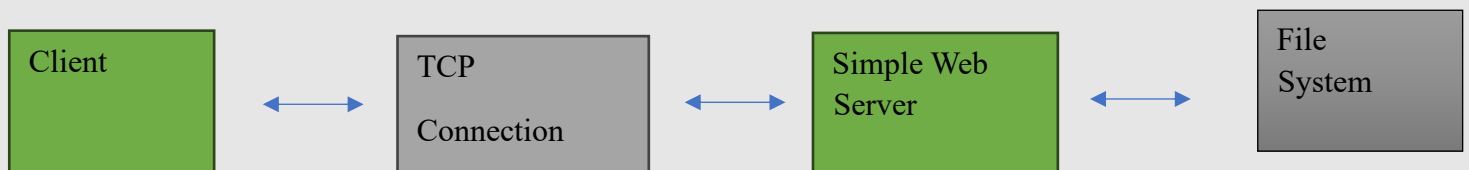
- The code and configuration should be well-documented to facilitate future updates, fixes, and enhancements

➤ **Compatibility:**

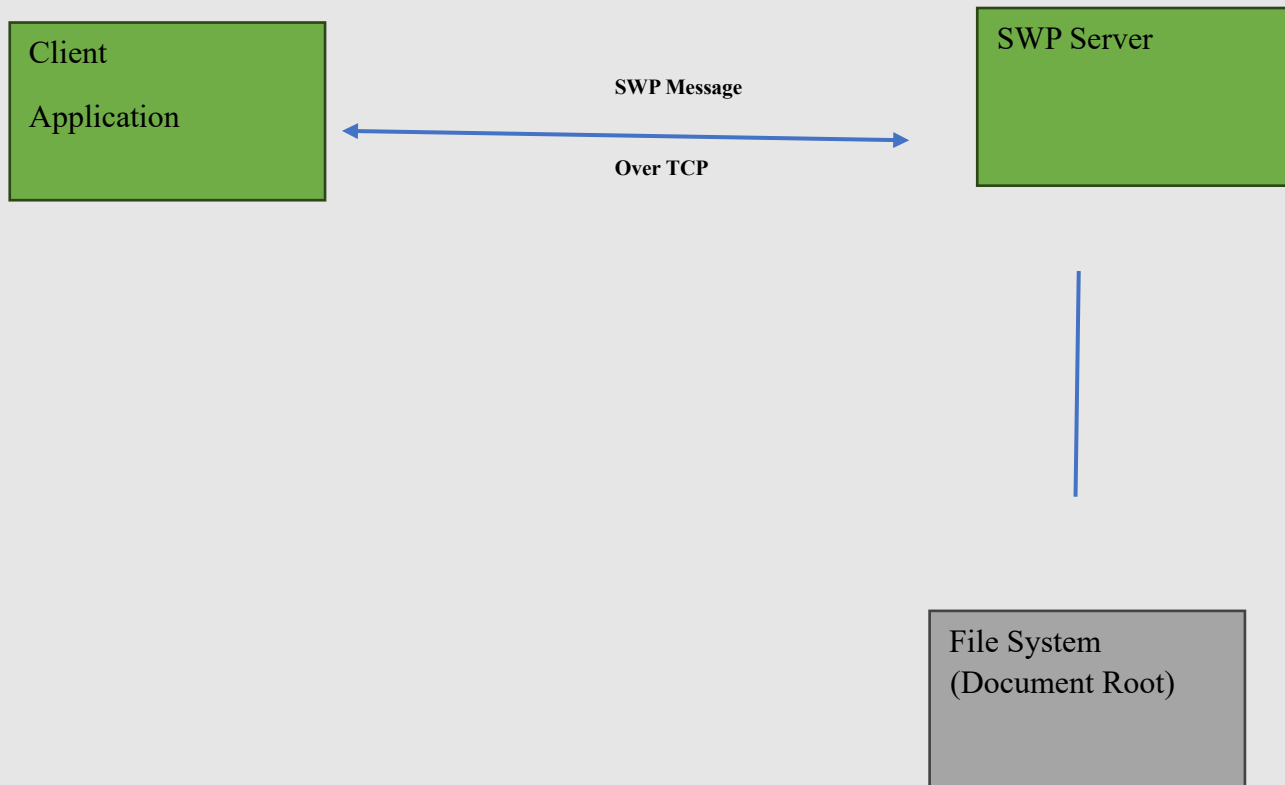
- The server must be compatible with standard web browsers and different operating systems (e.g., Windows, Linux)

Application Architecture

System Overview:



Component Architecture



Connection Flow

1. **Connection Establishment:** Client opens TCP connection to server (default port 8080)
2. **Request Phase:** Client sends SWP request message
3. **Processing Phase:** Server parses request, locates resource, generates response
4. **Response Phase:** Server sends SWP response message
5. **Connection Management:** Connection closed or kept open based on headers

Message Types

Request message

- Initiated by client to request a resource or perform an action.
- **Request Line:** Contains the HTTP method (e.g., GET, POST, PUT, DELETE), the URL of the requested resource, and the HTTP version
- **Header Fields:** Provide additional information about the request, such as the client's capabilities, preferred language, or authentication credentials
- **Empty Line:** Separates the header fields from the message body
- **Message Body (optional):** Contains data being sent to the server, typically used with methods like POST or PUT for submitting forms or uploading files

Response Message

- Server's reply to a client request, containing status and optional data.
- **Status Line:** Contains the HTTP version, a three-digit HTTP status code (e.g., 200 OK, 404 Not Found, 500 Internal Server Error), and a textual reason phrase describing the status
- **Header Fields:** Provide information about the response, such as the content type, content length, caching directives, or cookies
- **Empty Line:** Separates the header fields from the message body
- **Message Body (optional):** Contains the requested resource (e.g., HTML, CSS, images, JSON data) or an error message

Message Format

Request Message Format

```
<method> SP <request-uri> SP <protocol-version> CRLF
<header-field-name> ":" <header-field-value> CRLF
... (additional headers) ...
CRLF
<message-body> (optional)
```

1. Start Line: <method> SP <request-uri> SP <protocol-version> CRLF

Method

- **Purpose:** Specifies the action to be performed on the resource
- **Values:** GET, HEAD, POST
- **Example:** GET

SP (Space)

- **Purpose:** Delimiter separating the components
- **Character:** ASCII space (0x20)
- **Usage:** Must be exactly one space character

Request-URI

- **Purpose:** Identifies the target resource on the server
- **Format:** Path to the resource starting with /
- **Examples:**
 - /index.html
 - /images/logo.png
 - /api/users

Protocol-Version

- **Purpose:** Specifies the protocol version for compatibility
- **Format:** SWP/1.0
- **Example:** SWP/1.0

CRLF (Carriage Return + Line Feed)

- **Purpose:** Line terminator marking the end of the start line
- **Characters:** \r\n (ASCII 13 + ASCII 10)

- **Importance:** This is crucial for proper parsing

2. Header Section: <header-field-name> ":" <header-field-value> CRLF

Header Structure

- **Format:** Name: Value
- **Case Sensitivity:** Header names are case-insensitive
- **Multiple Headers:** Each header on its own line

The Colon (:)

- **Purpose:** Separates header name from value
- **Position:** Must immediately follow header name
- **Spacing:** Optional space after colon (recommended for readability)

3. Empty Line: CRLF

Purpose

- **Signal:** Marks the end of the headers section
- **Format:** A single line containing only \r\n
- **Importance:** This is MANDATORY - it tells the parser that headers are complete and the body may follow

4. Message Body: <message-body> (optional)

When Present

- **GET/HEAD requests:** No body allowed
- **POST requests:** Contains data to be processed
- **Format:** Raw binary or text data

Body Length

- Determined by:

1. Content-Length header (explicit size)
2. Connection close (implicit - read until connection ends)

Response Message Format

```
<protocol-version> SP <status-code> SP <reason-phrase> CRLF  
<header-field-name> ":" <header-field-value> CRLF  
... (additional headers) ...  
CRLF  
<message-body> (optional)
```

1. Status Line: <protocol-version> SP <status-code> SP <reason-phrase> CRLF

Protocol-Version

- **Purpose:** Indicates the SWP version the server is using
- **Format:** SWP/1.0
- **Example:** SWP/1.0
- **Importance:** Helps client understand protocol capabilities

SP (Space)

- **Purpose:** Delimiter separating the components
- **Character:** ASCII space (0x20)
- **Usage:** Must be exactly one space character between each component

Status-Code

- **Purpose:** 3-digit numerical code indicating request outcome
- **Format:** Three digits (e.g., 200, 404, 500)
- **Categories:**
 - **1xx:** Informational (reserved for future use)
 - **2xx:** Success
 - **3xx:** Redirection (reserved for future use)
 - **4xx:** Client Error
 - **5xx:** Server Error

Reason-Phrase

- **Purpose:** Human-readable description of the status code
- **Format:** Text description
- **Examples:**
 - OK
 - Not Found
 - Internal Server Error
- **Importance:** Helps humans understand the status quickly

CRLF (Carriage Return + Line Feed)

- **Purpose:** Line terminator marking the end of the status line
- **Characters:** \r\n (ASCII 13 + ASCII 10)

2. Header Section: <header-field-name> ":" <header-field-value> CRLF

Header Structure

- **Format:** Name: Value

- **Case Sensitivity:** Header names are case-insensitive
- **Multiple Headers:** Each header on its own line

The Colon (:

- **Purpose:** Separates header name from value
- **Position:** Must immediately follow header name
- **Spacing:** Optional space after colon (recommended for readability)

3. Empty Line: CRLF

Purpose

- **Signal:** Marks the end of the headers section
- **Format:** A single line containing only `\r\n`
- **Importance:** This is MANDATORY - it tells the parser that headers are complete and the body may follow

4. Message Body: <message-body> (optional)

When Present

- **Successful requests:** Contains the requested resource
- **Error responses:** May contain error details or HTML error page
- **HEAD requests:** Never contains a body

Body Content Types:

- **HTML pages:** text/html
- **CSS files:** text/css
- **Images:** image/jpeg, image/png, etc.
- **JSON data:** application/json
- **Plain text:** text/plain

Body Length

- **Determined by:** Content-Length header (explicit size)
- **Must match:** Actual number of bytes in body

Any Additional Information for the Developers

- **Connection Management:** Specify if your protocol uses a **non-persistent** (one request per connection) or **persistent** connection (multiple requests per connection). For example: "SWP/1.0 uses non-persistent connections by default. The connection is closed by the server after sending the response."
- **Error Handling:** List common status codes and what they mean (e.g., 400 Bad Request, 403 Forbidden, 404 Not Found, 500 Internal Server Error). Explain what a developer should log when these occur.
- **State Management:** Since your protocol is stateless, briefly mention that if state is needed (e.g., user login), it must be managed by the application itself using techniques like cookies (which would be implemented as custom headers).
- **Security Considerations (Bonus):** Mention that your base protocol (SWP) does not include encryption. Suggest that for security, it should be layered over TLS/SSL (becoming SWPS, akin to HTTPS).
- **Configuration Files:** Developers need to know how to edit configuration files to customize server settings. Apache uses a decentralized approach with .htaccess files, while Nginx prefers a centralized configuration.
- **Logging:** Web servers produce detailed log files (access logs, error logs) that are essential for debugging, monitoring traffic, and security analysis.
- **Security Measures:** Developers should implement security best practices such as using HTTPS (via SSL/TLS certificates), configuring firewalls (Web Application Firewalls), and keeping software updated with security patches.

- **Performance Optimization:** Techniques like **caching** (storing copies of frequently accessed files), **compression** (reducing file size), and **load balancing** (distributing traffic across multiple servers) are crucial for fast response times and scalability.
- **Virtual Hosting:** This feature allows a single physical server to host multiple websites (domains), which is standard practice in most hosting environments.